AI COURSEWORK DESIGN AND DEVELOPMENT OF SPECIALIZED ARTIFICIAL NEURAL NETWORKS FOR INDUSTRY-SPECIFIC APPLICATIONS

Explainable Artificial Neural Network for Industrial Anomaly Detection and Defect Localization Using the MVTec AD Hazelnut Dataset

Google Drive Folder Link - https://drive.google.com/drive/folders/1-rV2YgdpNL7NsuR3KYBQkkZ-KzH0YiGE?usp=drive_link

Github Repository Link - https://github.com/NipuliniJ/Hazelnut-Anomaly-Detection

Nipulini Tanjala Jayasinghe

COMSCDS252P-011

Table of Contents

Question 1 6

1.1 Dataset description 6

1.1.1 Introduction & Background 6

1.1.2 Problem Statement 6

1.1.3 Dataset Selection 6

1.1.4 Dataset Structure 6

1.1.5 Dataset Characteristics 7

1.2 EDA 7

1.2.1 Class Distribution Analysis 7

1.2.2 Visual Inspection of Images 7

1.2.3 Defect Category Comparison 8

1.2.4 Image Properties and Pixel Intensity Analysis 8

1.2.5 Ground-Truth Mask Analysis 8

1.2.6 Defect Pixel Distribution 8

1.3 Consequences of False Positives and False Negatives 9

1.4 Selection and Justification of Evaluation Metrics 9

1.4.1 Image-level evaluation 9

1.4.2 Pixel-level evaluation 9

1.5 Data Quality and Data Leakage 10

1.5.1 Initial Data Preparation 10

1.5.2 Data Quality and Preprocessing 10

1.5.3 Data Leakage Prevention 10

1.6 Three Key EDA Findings and Their Modelling Implications 11

1.6.1 Methodology 11

1.6.1.1 Selected Learning Approach 11

1.6.1.2 Proposed ANN Architecture 11

1.6.2 Imbalanced dataset with a limited number of defective samples 11

1.6.3 Defects vary in appearance and may occupy only small image regions 12

1.6.4 Normal hazelnuts exhibit natural visual variation 12
Question 2 12
2.a ANN Architecture and Design 12
2.a.1 Design and Architecture of the ANN 12
2.a.1.1 Input 12
2.a.1.2 Encoder 13
2.a.1.3 Decoder 13
2.a.1.4 Architecture Summary 14
2.a.2 Architectural Assumptions and Suitability for the Problem 14
2.a.2.1 Spatial Patterns as Important Features 14
2.a.2.2 Visual Features Across Different Image Locations 14
2.a.2.3 Learnable Normal Visual Patterns 14
2.a.2.4 Consistent Dimensions and Colour Information 15
2.a.2.5 Compressed Representation of Visual Characteristics 15
2.a.3 Justification of Major Design Decisions 15
2.a.3.1 Network Depth 15
2.a.3.2 Network Width 15
2.a.3.3 Normalisation 16
2.a.3.4 Regularization 16
2.a.3.5 Output Layer 16
2.a.3.6 Loss Function 17
2.b Discussion of Alternative Approaches 17
2.b.1 Classical Machine Learning – Isolation Forest 17
2.b.2 Supervised CNN Classification 17
2.b.3 CNN Autoencoder – Selected Approach 18
2.b.4 Comparison and Justification of the Selected Approach 18
2.c Suitability of the ANN and Expected Performance 19
2.c.1 Why an ANN Is Appropriate for This Problem 19
2.c.2 Expected Performance Before Evaluating the Final Results 19

2.c.3 Alternative Approaches Considered 20
2.c.3.1 Isolation Forest – Classical Machine-Learning Alternative 20
2.c.3.2 Supervised CNN Classification 20
2.c.4 Overall Justification 20
Question 3 21
3.a Development, Baseline Comparison and Error Analysis 21
3.a.1 Initial Neural Model and Development Process 21
3.a.2 Learning Curves and Model Behaviour 22
3.a.3 Classical Machine-Learning Baseline 23
3.a.4 Model Comparison 24
3.a.5 Error Analysis of the Final Model 24
3.a.5.1 False Positives 25
3.a.5.2 False Negatives 25
3.a.5.3 Error Patterns and Limitations 25
3.b In-depth Study of Optimisation Techniques 26
3.b.1 Effect of Learning Rate 26
3.b.2 Effect of Network Depth and Bottleneck Size 26
3.b.3 Optimiser Comparison: Adam and SGD 27
3.b.4 Unsuccessful Approaches and Diagnosis 27
3.b.4.1 Lower Learning Rates with Adam 28
3.b.4.2 SGD Optimiser Configurations 28
3.b.4.3 Smaller V3 Bottleneck 28
3.b.5 Overall Findings 28
Question 4 – Web Application Implementation 28
4.1 Application Overview and Architecture 29
4.2 Prediction Workflow 29
4.3 Prediction Score and Interpretation 29
4.4 Limitations Communicated to Users 30
4.5 Application Testing 30

Question 5 30
5.a Discussion of XAI Methodologies 31
5.a.1 Reconstruction-Error Heatmaps – Architecture-Specific Approach 31
5.a.2 LIME – Model-Agnostic Approach 31
5.a.3 SHAP – Model-Agnostic Approach 31
5.a.4 Comparison of XAI Approaches 31
5.a.5 Justification of the Selected Method 32
5.b Implementation and Validation of XAI 33
5.b.1 Implementation of the Explanation Method 33
5.b.2 Integration into the Web Application 34
5.b.3 Sanity-Checking the Explanations Using Occlusion 34
5.b.4 Analysis of Incorrect Predictions 34
5.b.4.1 False Positive – 309.png 35
5.b.4.2 False Negative – 005.png 35
5.b.4.3 False Negative – 003.png 35
5.b.5 Limitations of the Explanation Method 35
Question 6 36
6.a Application of a Pretrained Foundation Model 36
6.a.1 Model Selection and Justification 36
6.a.2 Model Configuration and Implementation 36
6.a.3 Prompt Design and Validation 36
6.a.4 Evaluation on the Held-Out Test Dataset 37
6.a.5 Comparison with the CNN Autoencoder 38
6.a.6 Computational Cost and Limitations 38
6.a.7 Summary 39
6.b Comparative Analysis and Innovation Opportunities 40
6.b.1 Comparative Predictive Performance 40
6.b.2 Inference Latency and Computational Cost 40
6.b.3 Training Data Requirements 41

6.b.3.1 CNN Autoencoder 41
6.b.3.2 CLIP Foundation Model 41
6.b.4 Privacy and Dependence on Third-Party Services 42
6.b.5 Conditions for Deployment 42
6.b.5.1 Conditions Favouring the CNN Autoencoder 42
6.b.5.2 Conditions Favouring CLIP 42
6.b.6 Innovation Opportunity: A Foundation-Model-Assisted Quality Inspection System 42
6.b.7 Overall Conclusion 43

Question 1

1.1 Dataset description

1.1.1 Introduction & Background

Quality checks are extremely important in the process of industrial manufacturing in order to detect flaws and reduce wastage. In traditional techniques, quality checks were manually performed which led to inconsistency and were a time-taking process as well. The application of Artificial Neural Networks (ANNs) such as Convolutional Neural Networks (CNNs) in visual inspections has helped in automating the process of detecting flaws in the products by analyzing their pictures. The present study focuses on the anomaly detection of hazelnuts through a CNN autoencoder. This project investigates the application of a CNN-based autoencoder for detecting and localising anomalies in hazelnut images using the MVTec Anomaly Detection (MVTec AD) dataset.

1.1.2 Problem Statement

Quality control systems in industries struggle with correctly identifying the products that are defective owing to the difficulties associated with acquiring enough labelled data. Although supervised models require such data, the proposed approach in this study involves training a convolutional neural network autoencoder only on normal hazelnuts to identify anomalies and mark abnormal areas.

1.1.3 Dataset Selection

The project uses the Hazelnut category of the MVTec Anomaly Detection (MVTec AD) dataset.

MVTec AD is an industrial anomaly detection dataset designed to support research into automated visual inspection. It contains normal product images and images containing different types of defects, together with corresponding ground truth segmentation masks for anomalous regions. The Hazelnut category was selected because it provides both normal and defective product images, allowing the investigation of image-level anomaly detection and pixel-level defect localization. Dataset source: https://www.mvtec.com/research-teaching/datasets/mvtec-ad

1.1.4 Dataset Structure

The selected Hazelnut dataset contains 501 images in total.

The original dataset is organised into three main directories:

- train: Contains normal hazelnut images.
- test: Contains normal and defective hazelnut images.

- ground_truth: Contains segmentation masks corresponding to defective test images.

The dataset distribution is summarised below.

Dataset directory	Category	Number of images
Train	Good	391
Test	Good	40
Test	Crack	18
Test	Cut	17
Test	Hole	18
Test	Print	17
Ground truth	Defect masks	70
Total images		**501**

The four defect categories represent different types of visual abnormalities:Crack, Cut, Hole &Print.Each defective test image has a corresponding ground-truth mask identifying the defective region.

1.1.5 Dataset Characteristics

The training set contains only normal hazelnuts, while the test set has the defective hazelnuts. This makes an excellent setting for one-class anomaly detection where the objective is to concentrate on the attributes of normal products rather than defects classification. The target variable is the binary class where the hazelnut is either normal (class 0) or defective (class 1). Moreover, there is ground-truth mask information available for the defective hazelnuts.

1.2 EDA

A comprehensive exploratory data analysis was conducted to understand the visual characteristics, distribution and defect patterns within the MVTec AD Hazelnut dataset. Since the dataset consists of RGB images with pixel-level defect annotations, the analysis focused on class distribution, visual inspection, image properties, pixel intensity distributions and defect localisation.

1.2.1 Class Distribution Analysis

The dataset contains 501 images, comprising 431 normal images and 70 defective images. The defective samples are distributed across four categories: crack (18), cut (17), hole (18) and print (17). This indicates a significant class imbalance, with normal samples

representing the majority of the dataset. This imbalance was considered when selecting evaluation metrics and designing the modelling approach.

1.2.2 Visual Inspection of Images

Representative normal and defective hazelnut images were visually inspected to identify differences in surface appearance. Normal samples exhibit natural variations in colour, texture and shape. Defective samples contain visible abnormalities, including cracks, cuts, holes and printing defects. The inspection also revealed that some defects occupy relatively small regions of the image, making them difficult to identify through image-level classification alone.

1.2.3 Defect Category Comparison

The four defect categories were compared visually to understand their differences in appearance, shape and spatial distribution. Cracks generally appear as narrow, irregular lines, while cuts and holes produce different localised surface abnormalities. Printing defects exhibit changes in surface appearance. These variations demonstrate that a single anomaly detection model must account for different types of visual irregularities.

1.2.4 Image Properties and Pixel Intensity Analysis

Image dimensions and pixel intensity distributions were examined to understand the input characteristics. The original images have a resolution of 1024 × 1024 pixels and contain three colour channels (RGB). Pixel intensity distributions were also investigated to observe variations in colour and brightness across the dataset. For model development, images were resized to 128 × 128 pixels and normalised to the range [0, 1].

1.2.5 Ground-Truth Mask Analysis

The ground-truth segmentation masks were examined to understand the location and shape of defective regions. The masks provide pixel-level annotations for defective samples, allowing the model's localisation performance to be evaluated separately from image-level classification performance. Overlay visualisations were also generated by combining the original images with their corresponding masks to assess how defects are distributed across the hazelnut surfaces.

1.2.6 Defect Pixel Distribution

The proportion of defective pixels was investigated across the four defect categories. The analysis showed variations in the proportion of image area occupied by defects, indicating that some abnormalities are considerably more localised than others. This is particularly

important because small defects may contribute only a limited amount to the overall image reconstruction error.

Overall, the EDA highlighted three important characteristics: class imbalance, substantial variation in normal surface appearance and differences in defect size and location. These findings influenced the decision to use a CNN-based autoencoder trained on normal images, together with reconstruction-error heatmaps for anomaly detection and defect localisation.

1.3 Consequences of False Positives and False Negatives

Both false negatives and false positives affect the operations and economy of food industries. False positive arises when a good nut is incorrectly classified as bad, causing unnecessary rejections and expenses. On the other hand, false negatives arise when a bad nut is correctly classified as good, which is dangerous to the quality of the food products and the reputation of the brand. Although false negatives are important to consider for quality control, it is necessary to ensure that false negatives are kept in check while minimizing false positives.

1.4 Selection and Justification of Evaluation Metrics

The evaluation metrics were selected to assess both image-level anomaly detection and pixel-level defect localisation. Since the dataset contains more normal images than defective images, accuracy alone would not provide a sufficient assessment of model performance.

1.4.1 Image-level evaluation

Average Precision (AP) is the key performance metric for evaluating the performance of anomaly detection in terms of images. It is important due to the unbalanced nature of the data set and helps ensure that defective items are recognized accurately for industrial quality control applications. Recall helps determine how many defects have been detected by the model, whereas precision helps assess the correctness of the detected defect. F1-score uses both metrics and helps solve the problem of missed defects and false rejection, and ROC-AUC is another criterion of the model's performance.

1.4.2 Pixel-level evaluation

Since the dataset provides ground-truth defect masks, additional metrics were used to evaluate defect localisation. Pixel-level Average Precision measures how effectively the model identifies defective pixels, while Dice coefficient and Intersection over Union (IoU) measure the spatial overlap between predicted defect regions and ground-truth masks.

Pixel-level ROC-AUC was also used to assess the model's ability to distinguish defective pixels from normal pixels.

Overall, the selected metrics provide a more comprehensive evaluation than accuracy alone. AP and recall assess the effectiveness of identifying defective products, while precision evaluates the reliability of defect alerts. Pixel-level metrics provide further insight into whether the model can correctly identify the location of defects.

1.5 Data Quality and Data Leakage

Several data preparation and validation measures were applied to maintain data quality and minimise the risk of data leakage throughout the project.

1.5.1 Initial Data Preparation

The dataset was downloaded from the official MVTec AD source and uploaded to the Google Colab environment.The archive was extracted, and the resulting directory structure was inspected to confirm the availability of the training images, test images and ground-truth masks.

The following initial data preparation activities have been completed:

- Verified the presence of the training, testing and ground-truth directories.
- Confirmed the image counts across the normal and defective categories.
- Checked the availability of corresponding ground-truth masks for defective images.
- Inspected representative normal and defective images.
- Confirmed that the images are RGB images with dimensions of 1024 × 1024 pixels.
- Successfully loaded images using PyTorch.
- Tested image resizing to 128 × 128 pixels and conversion into tensors.

These initial checks confirmed that the dataset could be accessed and processed using the selected Python environment.

1.5.2 Data Quality and Preprocessing

The MVTec AD Hazelnut dataset has been examined to find visual features and the type of defects. Original RGB images of size 1024 × 1024 have been rescaled to 128 × 128 pixels where pixel intensities lie within the range of [0, 1]. Rescaling of the ground truth segmentation masks has been done using the nearest neighbour interpolation method. Dataset manifest file has been prepared for organizing information about files.

1.5.3 Data Leakage Prevention

A custom training, validation and test split was created using a fixed random seed (42). The 391 normal images were divided into 313 training images, 59 validation images and 19 test images. However, the original test set also contained 40 normal images and 70 defective images. Therefore, the final split consisted of 313 training images, 94 validation images and 94 test images, with the validation and test sets each containing 59 normal and 35 defective images.

The manifest for the dataset was checked in order to confirm that there were no overlapping images between the train, validation, and test splits with all the image paths required. In this case, only the images with normal objects were used for the training phase without using the images with defects in order to study normal reconstruction. The validation dataset was used for selection and threshold tuning, and the test set was employed exclusively for performance testing. The threshold for classification was determined using the validation set in order not to influence the test set.

1.6 Three Key EDA Findings and Their Modelling Implications

1.6.1 Methodology

1.6.1.1 Selected Learning Approach

The project utilizes a one-class anomaly detection approach rather than a supervised classification approach since the dataset has 391 good images and only 70 bad images, each divided into four categories. A supervised learning algorithm requires a combination of good and bad images to function, but the limited number of bad images will affect its learning ability. The one-class anomaly detection method will enable the algorithm to learn how to detect anomalies from normal objects.

1.6.1.2 Proposed ANN Architecture

A Convolutional Neural Network (CNN)-based autoencoder is proposed as the primary artificial neural network.An autoencoder consists of two main components:

Encoder: Extracts and compresses visual information from the input image into a lower-dimensional representation.
Decoder: Reconstructs the input image from the compressed representation.
The model will be trained using normal hazelnut images. During inference, the input image will be reconstructed, and differences between the original image and its reconstruction will be investigated as potential indicators of anomalies.The reconstruction error will be used to generate anomaly scores and visual heatmaps.

1.6.2 Imbalanced dataset with a limited number of defective samples

The dataset contains 431 normal images and only 70 defective images. This imbalance makes conventional supervised classification challenging, as a model could become biased towards predicting the majority normal class. Therefore, an unsupervised anomaly-detection approach was selected. The CNN autoencoder was trained exclusively on normal hazelnut images so that it could learn normal appearance patterns and identify defects through reconstruction errors. Average Precision (AP), alongside precision and recall, was used to evaluate image-level detection rather than relying on accuracy alone.

1.6.3 Defects vary in appearance and may occupy only small image regions

Visual inspection showed that defects such as cracks, cuts, holes and print abnormalities differ in appearance, size and location. Some defects affect only a relatively small portion of the hazelnut surface. This influenced the selection of a CNN-based autoencoder, which can learn spatial features from image data. Reconstruction-error heatmaps were also generated to identify regions with unusually high reconstruction errors. For image-level detection, the average of the highest 1% of pixel reconstruction errors (Top-1 scoring) was selected on the validation set to make the score more sensitive to localised defects.

1.6.4 Normal hazelnuts exhibit natural visual variation

The regular images show variability in their colors, textures, shapes, and surfaces, which could result in the generation of false positives when the errors in the image reconstruction are misclassified as defects. Therefore, it is important to employ validation-based threshold setting rather than assuming that all errors are defects. The evaluation of both precision and recall allows balancing between false alarms and missed defects.

Question 2

2.a ANN Architecture and Design

2.a.1 Design and Architecture of the ANN

A Convolutional Autoencoder (CNN Autoencoder) was designed for unsupervised industrial anomaly detection using the MVTec AD Hazelnut dataset. The model learns the normal visual characteristics of hazelnut images by reconstructing them. During inference, defects are identified through differences between the original images and their reconstructed outputs. The architecture consists of an encoder and a decoder.

2.a.1.1 Input

Input: Each RGB image is resized to 128 × 128 pixels and normalised to the range [0, 1]. Therefore, the input tensor has dimensions **3 × 128 × 128**.

2.a.1.2 Encoder

The encoder extracts hierarchical spatial features and progressively reduces the spatial dimensions of the input. It contains four convolutional layers, each followed by a ReLU activation and 2 × 2 max-pooling operation.

Layer	Operation	Output dimensions
Input	RGB image	3 × 128 × 128
1	Conv2D (3 → 16), ReLU, MaxPool2D	16 × 64 × 64
2	Conv2D (16 → 32), ReLU, MaxPool2D	32 × 32 × 32
3	Conv2D (32 → 64), ReLU, MaxPool2D	64 × 16 × 16
4	Conv2D (64 → 128), ReLU, MaxPool2D	128 × 8 × 8

The resulting bottleneck representation is **128 × 8 × 8**, which contains the compressed visual features learned by the encoder.

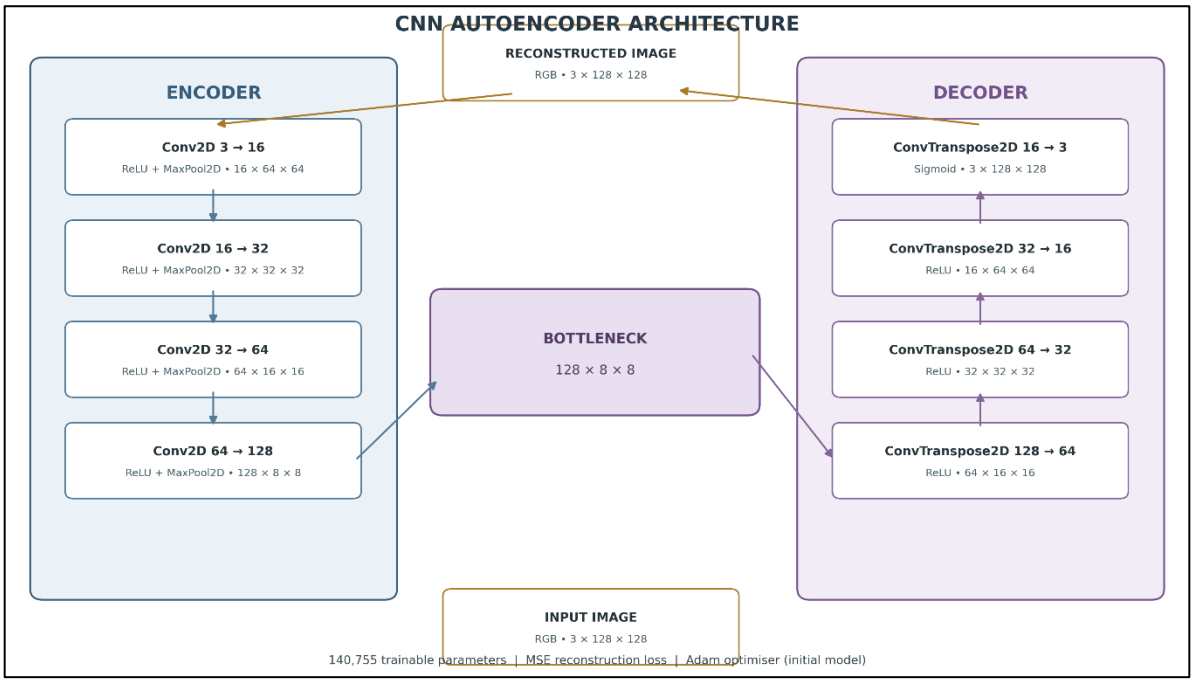
2.a.1.3 Decoder

The decoder reconstructs the original image from the compressed representation. It uses four transposed convolutional layers with a stride of 2 to progressively increase the spatial dimensions. ReLU activations are applied between the layers, while a Sigmoid activation is used in the final layer to produce pixel values within [0, 1].

Layer	Operation	Output dimensions
1	ConvTranspose2D (128 → 64), ReLU	64 × 16 × 16
2	ConvTranspose2D (64 → 32), ReLU	32 × 32 × 32
3	ConvTranspose2D (32 → 16), ReLU	16 × 64 × 64
4	ConvTranspose2D (16 → 3), Sigmoid	3 × 128 × 128

2.a.1.4 Architecture Summary

The network comprises 140,755 parameters and uses Mean Squared Error (MSE) for loss computation, measuring the error between the input images and the output images produced by the autoencoder. The Adam optimizer is used with a learning rate of 0.001 and a batch size of 8. The neural network works effectively since the convolutional layers help capture spatial relationships in the data, while the encoder-decoder helps learn the representation of the normal images.



2.a.2 Architectural Assumptions and Suitability for the Problem

The CNN autoencoder makes several assumptions about the input data and the nature of industrial defects.

2.a.2.1 Spatial Patterns as Important Features

The architecture assumes that neighboring pixels contain meaningful information about visual characteristics such as surface texture, edges, cracks and holes. Convolutional layers are suitable because they learn local spatial patterns using shared filters. This is appropriate for hazelnut inspection, where defects often appear as local changes in surface appearance.

2.a.2.2 Visual Features Across Different Image Locations

Convolutional layers assume that useful visual patterns can be recognised regardless of their exact position within an image. Weight sharing allows the same filters to detect features

across different spatial locations. This is useful because defects may occur in different regions of a hazelnut rather than at a fixed position.

2.a.2.3 Learnable Normal Visual Patterns

The autoencoder assumes that normal hazelnut images contain recurring visual characteristics that can be learned and reconstructed. Since the model is trained exclusively on normal images, it learns to reproduce these patterns. Defective regions are expected to produce larger reconstruction errors when their appearance differs from the learned normal patterns. This makes the architecture suitable for unsupervised anomaly detection.

2.a.2.4 Consistent Dimensions and Colour Information

The model assumes that all input images can be represented using three RGB channels and a fixed resolution of 128 × 128 pixels. Resizing and normalisation provide consistent inputs for training. Retaining RGB information also allows the model to learn colour and surface variations that may help distinguish normal from defective hazelnuts.

2.a.2.5 Compressed Representation of Visual Characteristics

The encoding process transforms the data into a 128 × 8 × 8 representation to ensure the preservation of vital visual information that is decoded by the decoder back into an image. It encourages pattern recognition rather than pixel-by-pixel memory. The technique can be used to detect anomalies in hazelnuts because the anomalies can be seen from the differences in texture, color, and structure of the hazelnuts. However, there are some disadvantages with the process, such as errors of reconstruction caused by image differences and minor anomalies that could be missed due to downsampling/pooling.

2.a.3 Justification of Major Design Decisions

The CNN autoencoder was designed to learn the visual characteristics of normal hazelnut images and identify potential defects through reconstruction errors. The following design decisions were made to support this objective.

2.a.3.1 Network Depth

The architecture of the network is such that there are four convolutional layers in the encoder and four transposed convolutional layers in the decoder. In this manner, there are several layers for feature extraction, where the basic features such as edges and textures are extracted at the early layers by the encoder and more complex features at deeper layers. In the end, the decoder creates the original image based on the compressed input. Although a deeper neural network can extract more complex features, it will also require higher computation and will overfit on a small dataset.

2.a.3.2 Network Width

The number of feature channels increases progressively from 3 to 16, 32, 64 and 128 in the encoder. This allows the network to learn a wider range of visual features as the spatial dimensions decrease.The decoder reverses this process, reducing the feature channels from 128 to 64, 32, 16 and finally 3 to reconstruct the RGB image.The 128-channel bottleneck provides sufficient feature capacity to represent normal hazelnut structures while keeping the model relatively compact. The complete architecture contains approximately 140,755 trainable parameters.

2.a.3.3 Normalisation

The input images are represented as RGB tensors and normalized by dividing their pixel values by 255, which puts them in the [0, 1] range. This allows us to have our input values in the same numeric range, facilitating the process of optimization and making it relevant to use a sigmoid output layer.Batch Normalization did not enter into the architecture of the model. The relatively small size of the training dataset and the simplicity of the convolutional autoencoder were taken into account when designing it.

2.a.3.4 Regularization

None of the regularization mechanisms such as Dropout or weight decay were used in the model's structure. The model was trained only on normal hazelnuts' pictures with monitoring the validation loss of reconstruction. This method can be justified by the idea of using the model for anomaly detection because the model is working with the patterns of normal samples but not the defects. Nevertheless, this can be an issue since the model may produce some defects during the reconstruction process.

2.a.3.5 Output Layer

The last layer of the decoder is done with a transposed convolution for creating an RGB image. The sigmoid function is used to ensure that the value of the pixels stays between [0, 1]. No classification is performed by the model. The anomaly is detected on the basis of differences in pixel-wise comparison of the original image and the reconstructed one.

2.a.3.6 Loss Function

The Mean Squared Error (MSE) is applied as the reconstruction loss to calculate the mean square error between the original hazelnut image and its corresponding reconstructed version. As MSE tends to give more weight to large differences, it is suitable to train the autoencoder in order to correctly reconstruct normal images. Only normal images will be used to train the model, and the reconstruction error will be used at the inference step to find anomalies.

2.b Discussion of Alternative Approaches

Several approaches can be considered for industrial anomaly detection. The selection of the CNN autoencoder was based on the characteristics of the MVTec AD Hazelnut dataset, the availability of training labels, the need to identify previously unseen defects, and the requirement to localise defective regions.

2.b.1 Classical Machine Learning – Isolation Forest

Isolation Forest is an unsupervised anomaly detection model that isolates anomalous data points with the help of decision trees. In this project, images in RGB format were rescaled to 32×32 pixels, and then converted to feature vectors to be fed into the model. It performed quite poorly with ROC-AUC of 0.7298 and Average Precision (AP) of 0.5372, thus acting as a baseline for anomaly detection task. It was outperformed by a CNN autoencoder with ROC-AUC of 0.8969 and AP of 0.8810. Besides, flattening images deprived the Isolation Forest of the ability to localize defects on a pixel level.

2.b.2 Supervised CNN Classification

A supervised CNN classifier is another possible approach. Unlike an autoencoder, a classifier would learn to distinguish between normal and defective hazelnut images using labelled examples. Its final output could represent the probability of an image belonging to the defective class.This approach could be effective when sufficient labelled examples are available for all relevant defect categories. However, the dataset contains only 70 defective images, compared with 431 normal images. The relatively small number of defective examples and the imbalance between classes could make it difficult for a supervised classifier to learn reliable representations of different defect types.

Furthermore, a conventional image-level classifier would identify whether an image is defective but would not automatically explain where the defect is located. Additional localisation mechanisms would be required.For these reasons, a supervised classifier was not selected as the primary architecture.

2.b.3 CNN Autoencoder – Selected Approach

The CNN autoencoder was selected because it supports unsupervised anomaly detection while preserving the spatial structure of images.

The encoder learns compressed representations of normal hazelnut images, while the decoder attempts to reconstruct them. During inference, regions with larger differences between the original and reconstructed images are treated as potential anomalies.

This approach was particularly suitable for the project for three reasons:

- **Limited defective training examples:** The model can be trained using normal images only, avoiding the need for a large collection of labelled defective examples during training.
- **Detection of different defect patterns:** Rather than learning only predefined defect categories, the model identifies deviations from the normal visual patterns learned during training.
- **Defect localisation:** Pixel-wise reconstruction errors can be visualised as anomaly heatmaps, helping identify the regions that contributed to an anomaly prediction.

The selected approach also supports the industrial requirement of identifying potentially defective products for further inspection rather than relying exclusively on a binary classification result.

2.b.4 Comparison and Justification of the Selected Approach

The alternatives differ in their training requirements, detection capabilities and suitability for defect localisation.

Approach	Training requirement	Main advantage	Main limitation
Isolation Forest	Normal training images with extracted features	Simple classical anomaly-detection baseline	Lower validation performance and no natural pixel-level localisation
Supervised CNN classifier	Labelled normal and defective images	Learns directly from known defect examples	Requires sufficient labelled defects and

			additional localisation mechanisms
CNN Autoencoder	Normal training images only	Learns normal patterns and produces reconstruction-error heatmaps	May reconstruct some defects or respond to natural variations in normal images

Based on the experimental results and the project requirements, the CNN autoencoder was selected as the primary approach. It provided stronger validation performance than the Isolation Forest baseline while also supporting pixel-level anomaly visualisation.

However, the autoencoder is not without limitations. It can produce high reconstruction errors for naturally varying normal regions and may fail to identify defects that it can reconstruct successfully. These limitations are considered in the subsequent evaluation and explainability sections.Therefore, the selected architecture represents a suitable balance between the available training data, anomaly-detection performance and the need for defect localisation in an industrial quality-inspection context.

2.c Suitability of the ANN and Expected Performance

2.c.1 Why an ANN Is Appropriate for This Problem

An Artificial Neural Network (ANN), specifically a Convolutional Neural Network (CNN) autoencoder, was selected based on the modality, volume, structure and objective of the dataset.

2.c.2 Expected Performance Before Evaluating the Final Results

Before examining the final test results, the CNN autoencoder was expected to outperform the classical Isolation Forest baseline on the primary image-level metric, Average Precision (AP).

The main reason for this expectation was the nature of the input data. The CNN can learn hierarchical spatial features directly from RGB images, whereas the Isolation Forest baseline uses flattened, resized image representations. Flattening the images removes their explicit two-dimensional spatial organisation, potentially limiting its ability to distinguish subtle surface defects from normal texture variations.Additionally, the CNN autoencoder learns a representation of normal hazelnut appearance through reconstruction. This is expected to help identify unusual visual patterns without requiring labelled examples of every defect category.

However, this expectation was not guaranteed. The dataset is relatively small, and classical machine-learning methods can perform effectively when training data is limited and the feature representation is informative. The CNN also introduces additional parameters and optimisation requirements, which could lead to overfitting or unstable generalisation.

Therefore, the expectation was that the CNN autoencoder would achieve higher AP because of its ability to learn spatial image features, while the Isolation Forest would provide a useful lower-complexity reference baseline. This prediction was made before comparing their final experimental performance.

2.c.3 Alternative Approaches Considered

2.c.3.1 Isolation Forest – Classical Machine-Learning Alternative

Isolation Forest is an unsupervised anomaly-detection algorithm that identifies unusual observations by isolating them through randomly generated decision trees. It is particularly appropriate for structured numerical data, where observations can be represented using meaningful features and the number of training samples may be limited.

For this project, resized RGB images were flattened into feature vectors and used to train the model on normal hazelnut images.However, the approach was considered less suitable as the primary solution because flattening the images removes their explicit spatial organisation. It also does not naturally generate pixel-level localisation maps. Since the project requires both anomaly detection and an indication of defect location, these limitations made Isolation Forest more appropriate as a classical baseline than as the final solution.

2.c.3.2 Supervised CNN Classification

A supervised CNN classifier would be appropriate if a sufficiently large and representative collection of labelled normal and defective images were available. It could learn to distinguish between the two classes directly and produce an image-level classification score.

However, this dataset contains only 70 defective images, distributed across four defect categories. This limited number of defective examples makes it challenging to train a reliable classifier that generalizes across different defect appearances. Furthermore, a conventional image-level classifier would not automatically provide precise defect localisation. Consequently, a supervised classifier was considered less suitable than an anomaly-detection approach trained using normal images only.

2.c.4 Overall Justification

The CNN autoencoder was selected because it provides a suitable combination of image-based feature learning, normal-only training and spatial anomaly visualisation. The Isolation Forest offers a useful classical reference, while supervised classification and traditional image processing have limitations arising from the dataset size, defect diversity and localisation requirements.The selected approach therefore aligns with the available data and the intended industrial application: identifying potentially defective hazelnuts and highlighting suspicious regions for further inspection.

Question 3

3.a Development, Baseline Comparison and Error Analysis

3.a.1 Initial Neural Model and Development Process

The initial neural model was a convolutional autoencoder designed to learn the visual characteristics of normal hazelnut images. It consisted of an encoder and a decoder, with four convolutional layers in the encoder and four transposed convolutional layers in the decoder.The initial model was trained using the 313 normal training images, with Mean Squared Error (MSE) as the reconstruction loss and the Adam optimiser with a learning rate of 0.001. The batch size was 8, and training was performed for a maximum of 30 epochs. Validation reconstruction loss was monitored during training.

The initial experiment achieved a best normal-validation reconstruction loss of approximately 0.000681 at epoch 30. Early stopping was not triggered.

Several experiments were subsequently conducted to investigate improvements to the architecture and optimisation process.

Experiment	**Modification**	**Validation reconstruction loss**
Experiment 1	Initial CNN autoencoder, Adam, learning rate 0.001	0.000681
Experiment 2	Reduced-depth V2 architecture, Adam, learning rate 0.001	0.000348
Experiment 3A	V2 architecture, Adam, learning rate 0.0005	0.000406
Experiment 3B	V2 architecture, Adam, learning rate 0.0001	0.000602
Experiment 4	Smaller bottleneck V3 architecture, Adam	0.000399
Experiment 5	V2 architecture, SGD, learning rate 0.01, momentum 0.9	0.011926

Experiment 6	V2 architecture, SGD, learning rate 0.001, momentum 0.9	0.050176

The experiments indicated that the V2 architecture achieved the lowest validation reconstruction loss among the tested configurations. However, reconstruction loss alone was not sufficient to determine the most appropriate final model because the project also required reliable image-level anomaly detection and defect localisation.

The experiments using SGD achieved substantially higher reconstruction losses than the Adam experiments under the tested 30-epoch configurations. This suggested that Adam was more effective for this particular training setup. However, the SGD experiments were still showing decreasing losses at the end of training, so the results do not establish that SGD would necessarily perform worse if given more training time or further tuning.

A separate evaluation compared three image-level anomaly-scoring methods using the same initial CNN autoencoder weights:

- **MSE scoring:** Average reconstruction error across all pixels.
- **Top-1% scoring:** Average reconstruction error across the highest-error 1% of pixels.
- **Top-5% scoring:** Average reconstruction error across the highest-error 5% of pixels.

The Top-1% method achieved the strongest validation Average Precision of 0.8810, compared with 0.7893 for MSE and 0.8261 for Top-5%. It was therefore selected as the final image-level anomaly-scoring method. This allowed the model to focus more strongly on localised regions with high reconstruction errors.

The final system consequently retained the initial CNN autoencoder architecture while adopting Top-1% scoring for image-level anomaly detection. This distinction is important: the improvement in scoring did not require changing the trained network weights.

3.a.2 Learning Curves and Model Behaviour

Training and validation reconstruction losses were monitored throughout the experiments to assess learning behaviour and identify potential underfitting or overfitting.

The initial CNN autoencoder achieved a best validation reconstruction loss of approximately 0.000681 at epoch 30. The reduced-depth V2 architecture achieved a lower validation reconstruction loss of approximately 0.000348, indicating that the smaller architecture was able to learn a more effective reconstruction function under the tested training conditions.

The learning curves should be interpreted by comparing the training and validation losses over successive epochs:

- **Underfitting:** Would be indicated by consistently high training and validation losses, suggesting that the model has insufficient capacity or has not learned the underlying normal image patterns effectively.
- **Overfitting:** Would be indicated by decreasing training loss accompanied by increasing validation loss, suggesting that the model is learning training-specific details that do not generalise to unseen normal images.
- **Continued optimisation:** Decreasing training and validation losses towards the final epoch suggest that the model is still improving and may benefit from additional training.

The SGD experiments showed continued decreases in validation loss by epoch 30, particularly indicating that the tested configurations had not clearly reached convergence. Therefore, their higher final losses should be interpreted in the context of the fixed training duration.Overall, the experiments did not provide sufficient evidence to conclude that the initial model was severely overfitting. However, the relatively small dataset and the reconstruction-based nature of the task remain limitations. A low reconstruction loss does not necessarily guarantee effective defect detection because a model may reconstruct some defective regions or produce high errors on naturally varying normal regions.

3.a.3 Classical Machine-Learning Baseline

Isolation Forest was implemented as the classical machine-learning baseline. It was trained using normal hazelnut images and provided a reference for evaluating whether the additional complexity of a CNN autoencoder offered a measurable benefit.

Before training, the RGB images were resized to 32 × 32 pixels and flattened into feature vectors containing 3,072 values per image.

The Isolation Forest configuration used:

- Number of estimators: 200
- Contamination: auto
- Random state: 42
- Parallel processing: enabled using all available CPU cores.

The model was trained using the 313 normal training images. Its validation performance was evaluated using image-level anomaly scores.

The Isolation Forest achieved a validation ROC-AUC of 0.7298 and Average Precision of 0.5372. Although it provided a useful reference, its performance was lower than that of the CNN autoencoder using Top-1% scoring.The main limitation was that flattening the images did not explicitly preserve their spatial structure. Furthermore, the approach did not naturally provide pixel-level defect localization.

Tuning search space: The reported Isolation Forest configuration is the implemented baseline configuration. A systematic multi-configuration hyperparameter search was not documented in the available experiment records. Therefore, the configuration should not be presented as the result of a search over multiple combinations unless additional experiment logs confirm this.

3.a.4 Model Comparison

Average Precision (AP) was selected as the primary image-level evaluation metric because the dataset contains substantially more normal images than defective images. AP evaluates the precision-recall trade-off across different anomaly-score thresholds and is therefore more informative than accuracy alone for this imbalanced dataset.

A constant-score reference benchmark can be included to represent a model with no ability to rank normal and defective images. With 35 defective images among 94 validation images, the reference AP is approximately 0.3723.

Model	**Primary metric: AP**	**Training time**	**Inference latency**
Constant-score reference benchmark	0.3723	Not applicable	Not applicable
Isolation Forest	0.5372	To be measured	To be measured
Initial CNN autoencoder (MSE scoring)	0.7893	To be measured	To be measured
Final CNN autoencoder (Top-1% scoring)	0.8810	To be measured	To be measured

The validation results show a clear improvement from the reference benchmark to the classical baseline and then to the CNN autoencoder. The final Top-1% scoring approach achieved the highest validation AP among these configurations.The comparison also demonstrates the benefit of concentrating on the regions with the largest reconstruction errors rather than averaging errors across the entire image. However, the results should be interpreted as validation performance; the final held-out test results are reported separately.

3.a.5 Error Analysis of the Final Model

The final CNN autoencoder was evaluated on 94 held-out test images using the Top-1% anomaly score and the threshold selected using the validation set.

The model achieved:

- ROC-AUC: 0.8877

- Average Precision: 0.8659
- Precision: 0.8214
- Recall: 0.6571
- F1-score: 0.7302
- Accuracy: 0.8191

The confusion matrix was:

	Predicted normal	**Predicted defective**
Actual normal	54	5
Actual defective	12	23

The model correctly classified 54 of the 59 normal images and 23 of the 35 defective images. However, it produced five false positives and twelve false negatives.

3.a.5.1 False Positives

Five normal hazelnut images were classified as defective. These errors may arise because normal hazelnuts contain natural variations in surface texture, colour and edges. Such variations can produce relatively high reconstruction errors even when no actual defect is present.For example, the normal image 309.png was incorrectly classified as defective, with an anomaly score of 0.025936. Its reconstruction-error heatmap can be examined to identify which regions contributed to the elevated score.

In an industrial inspection environment, these errors could lead to unnecessary manual inspections or rejection of acceptable products.

3.a.5.2 False Negatives

Twelve defective images were classified as normal. The model therefore detected 23 of the 35 defective images, corresponding to a defect recall of approximately 65.7%.

Two examples were 005.png and 003.png, both belonging to the crack category. Their anomaly scores were below the selected classification threshold, despite containing visible defects.This suggests that some defects may not produce sufficiently large reconstruction errors to cross the image-level decision threshold. Small or subtle defects may also contribute only a limited proportion of the total image error.

In an industrial setting, false negatives are particularly concerning because defective products could pass inspection.

3.a.5.3 Error Patterns and Limitations

The error analysis indicates that the model's performance depends not only on whether a defect is present but also on the extent and strength of its reconstruction error.

The Top-1% scoring method emphasises the highest-error regions, which helps detect localised defects. However, it can also respond to naturally unusual surface regions in normal images. Conversely, defects that are subtle or reconstructed relatively accurately may remain below the classification threshold.

The reconstruction-error heatmaps and incorrectly classified examples are examined further in the explainability analysis in Question 5.Overall, the model demonstrates promising anomaly-detection performance, but the false-negative results indicate that further improvements are necessary before it could be considered suitable for unsupervised industrial quality inspection.

3.b In-depth Study of Optimisation Techniques

The optimisation experiments investigated the effects of learning rate, network architecture and optimiser selection on the CNN autoencoder. Experiments were conducted using a maximum of 30 epochs, with batch size fixed at 8. Validation reconstruction loss was used to compare the training configurations.

3.b.1 Effect of Learning Rate

The learning rate controls the size of parameter updates during training. A rate that is too high may cause unstable updates, while a rate that is too low may result in slow convergence.

Three learning rates were investigated using the V2 architecture and Adam optimiser.

Experiment	Learning rate	Best validation reconstruction loss
Experiment 2	0.001	0.000348
Experiment 3A	0.0005	0.000406
Experiment 3B	0.0001	0.000602

The learning rate of 0.001 achieved the lowest validation reconstruction loss. Reducing the learning rate to 0.0005 and 0.0001 resulted in progressively higher losses within the 30-epoch training period.

This suggests that the lower learning rates were less effective under the available training duration, potentially because their smaller parameter updates required more epochs to reach a comparable solution. Therefore, 0.001 was retained as the preferred learning rate for the subsequent experiments.

3.b.2 Effect of Network Depth and Bottleneck Size

The architecture was modified to investigate whether reducing model complexity could improve reconstruction performance.

Architecture	Trainable parameters	Best validation reconstruction loss
V1 – Initial architecture	140,755	0.000681
V2 – Reduced-depth architecture	134,691	0.000348
V3 – Smaller bottleneck architecture	81,379	0.000399

V2 achieved the lowest validation reconstruction loss, despite having fewer parameters than the initial architecture. This indicates that increasing network capacity did not necessarily improve reconstruction performance for this dataset.

V3 reduced the parameter count by approximately 39.6% compared with V2. However, its validation loss was slightly higher. This suggests that further reducing the bottleneck capacity introduced some loss of reconstruction capability.

Consequently, V2 provided the best balance between model complexity and reconstruction performance among the tested architectures.

3.b.3 Optimiser Comparison: Adam and SGD

Adam and Stochastic Gradient Descent (SGD) were compared using the same V2 architecture.

Adam adapts parameter-specific learning rates using estimates of first and second moments of gradients. This can support efficient optimisation when different network parameters require different update magnitudes.

SGD updates parameters using gradient information and a specified learning rate. Momentum was used to accumulate information from previous gradients, helping maintain progress in consistent directions.

Optimiser	Learning rate	Momentum	Best validation reconstruction loss
Adam	0.001	Not applicable	0.000348
SGD	0.01	0.9	0.011926
SGD	0.001	0.9	0.050176

Adam achieved substantially lower validation reconstruction loss than both SGD configurations. The comparison at a learning rate of 0.001 is particularly useful because the architecture, training duration and learning rate were held constant.

The SGD learning curves continued to decrease towards the end of training, suggesting that the models had not necessarily reached convergence within 30 epochs. Therefore, the results indicate that Adam was more effective under the tested configurations rather than proving that it is universally superior to SGD.

3.b.4 Unsuccessful Approaches and Diagnosis

Two approaches did not produce improvements under the tested conditions.

3.b.4.1 Lower Learning Rates with Adam

Reducing the learning rate from 0.001 to 0.0005 and 0.0001 increased validation reconstruction loss from 0.000348 to 0.000406 and 0.000602, respectively.

The likely explanation is that smaller parameter updates slowed optimisation, preventing the model from reaching an equally effective solution within the fixed 30-epoch training period. These experiments demonstrate that a lower learning rate does not automatically improve model performance.

3.b.4.2 SGD Optimiser Configurations

Both SGD configurations produced considerably higher validation reconstruction losses than Adam. The learning curves indicated continued optimisation at epoch 30, suggesting that the selected learning rates and training duration were not sufficient to achieve comparable convergence.

Although SGD with momentum is a valid optimisation method, it was less effective than Adam for this particular dataset and experimental setup. Further tuning of the learning rate, momentum and training duration would be required before ruling it out completely.

3.b.4.3 Smaller V3 Bottleneck

The smaller V3 bottleneck also failed to outperform V2, indicating a trade-off between reducing model complexity and retaining sufficient representational capacity.

3.b.5 Overall Findings

The experiments support three main conclusions:

- **Learning rate:** Adam with a learning rate of 0.001 achieved the best result among the tested learning rates.
- **Architecture:** V2 achieved the lowest validation reconstruction loss, while further reducing the bottleneck size produced a small performance decline.
- **Optimiser:** Adam outperformed the tested SGD configurations within the 30-epoch training period.

Based on these findings, the V2 architecture with Adam at a learning rate of 0.001 was selected as the strongest reconstruction configuration. The experiments also demonstrate that optimisation decisions must be evaluated using validation evidence rather than assumptions about which configuration should perform best.

Question 4 – Web Application Implementation

4.1 Application Overview and Architecture

A web-based application was developed using Streamlit to demonstrate the practical application of the trained CNN autoencoder in an industrial quality-inspection scenario.

The intended users are production quality inspectors and shift supervisors who need to identify potentially defective hazelnuts and examine suspicious regions before making quality-control decisions.

The application consists of two main components:

- **Frontend:** A Streamlit interface that allows users to upload hazelnut images and view prediction results.
- **Inference component:** A Python-based model implementation that loads the trained CNN autoencoder checkpoint, processes the uploaded image and generates predictions at request time.

The application uses the trained model checkpoint, cnn_autoencoder_experiment_1.pth, and the same architecture implemented during model development.

4.2 Prediction Workflow

When a user uploads an image, the application performs the following operations:

1. Validates the uploaded image and converts it into RGB format.
2. Resizes the image to 128 × 128 pixels.
3. Normalises the pixel values to the range [0, 1].
4. Passes the image through the trained CNN autoencoder.
5. Compares the reconstructed image with the original image to calculate pixel-wise reconstruction errors.
6. Calculates the image-level anomaly score using the average of the highest-error 1% of pixels.
7. Compares the score with the validation-selected threshold to generate the final prediction.
8. Generates an anomaly heatmap and predicted defect-region visualisation.

The application performs inference using the trained model whenever an image is submitted. Predictions are not retrieved from precomputed results or stored test-set labels.

4.3 Prediction Score and Interpretation

The application displays an anomaly score rather than a conventional classification probability.The score represents the average reconstruction error across the highest-error 1% of image pixels. A higher score indicates that a larger reconstruction error is concentrated in the most anomalous regions.

The image is classified as defective when its anomaly score meets or exceeds the validation-selected threshold of -Threshold = 0.0192041565Otherwise, it is classified as normal.This score is not a calibrated probability or a guaranteed measure of confidence. It indicates the degree of reconstruction error relative to the selected decision threshold.The application also displays a reconstruction-error heatmap to help users identify the regions contributing to the anomaly score.

4.4 Limitations Communicated to Users

The application is a prototype for demonstrating anomaly detection and should not be treated as an autonomous quality-control system.

Its main limitations include:

- **Limited training data:** The model was trained using a relatively small collection of normal hazelnut images.
- **Unseen defect types:** Performance on defect patterns outside the training and evaluation distribution is not guaranteed.
- **False positives:** Natural variations in normal hazelnut texture and appearance may produce high reconstruction errors.
- **False negatives:** Some genuine defects may be reconstructed sufficiently well to remain below the classification threshold.
- **Threshold dependence:** Predictions depend on the threshold selected using the validation dataset.
- **Localisation limitations:** Heatmaps indicate reconstruction-error regions and should not be interpreted as perfectly accurate defect boundaries.

4.5 Application Testing

The application was tested using different input conditions to examine its prediction functionality and input-handling behavior. (As we have alimited pages to submit in the report answers for this section is explained and Included in the video uploaded)

Question 5

5.a Discussion of XAI Methodologies

Explainable Artificial Intelligence (XAI) is important in industrial quality inspection because users need more than a normal or defective prediction. They also need an indication of which image regions contributed to the model's decision.Three approaches were considered: reconstruction-error heatmaps, Local Interpretable Model-Agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP).

5.a.1 Reconstruction-Error Heatmaps – Architecture-Specific Approach

Reconstruction-error heatmaps were selected as the primary explanation method for this project. They are directly derived from the CNN autoencoder's reconstruction process.The method calculates the pixel-wise difference between the original image and its reconstructed version. Larger differences are represented by higher heatmap intensities, highlighting regions that the model reconstructed less accurately.

5.a.2 LIME – Model-Agnostic Approach

Local Interpretable Model-Agnostic Explanations (LIME) explains individual predictions by examining how changes to different parts of an input affect the model's output.For image classification, LIME divides an image into visually meaningful regions called superpixels. It then creates perturbed versions of the image by hiding selected regions and observes how the prediction score changes. A simpler interpretable model is fitted to approximate the original model's behaviour locally.

5.a.3 SHAP – Model-Agnostic Approach

SHapley Additive exPlanations (SHAP) uses concepts from cooperative game theory to estimate how individual input features contribute to a model prediction.

For image data, features can represent pixels or groups of pixels. SHAP estimates how the model output changes when different combinations of image regions are included or excluded.

5.a.4 Comparison of XAI Approaches

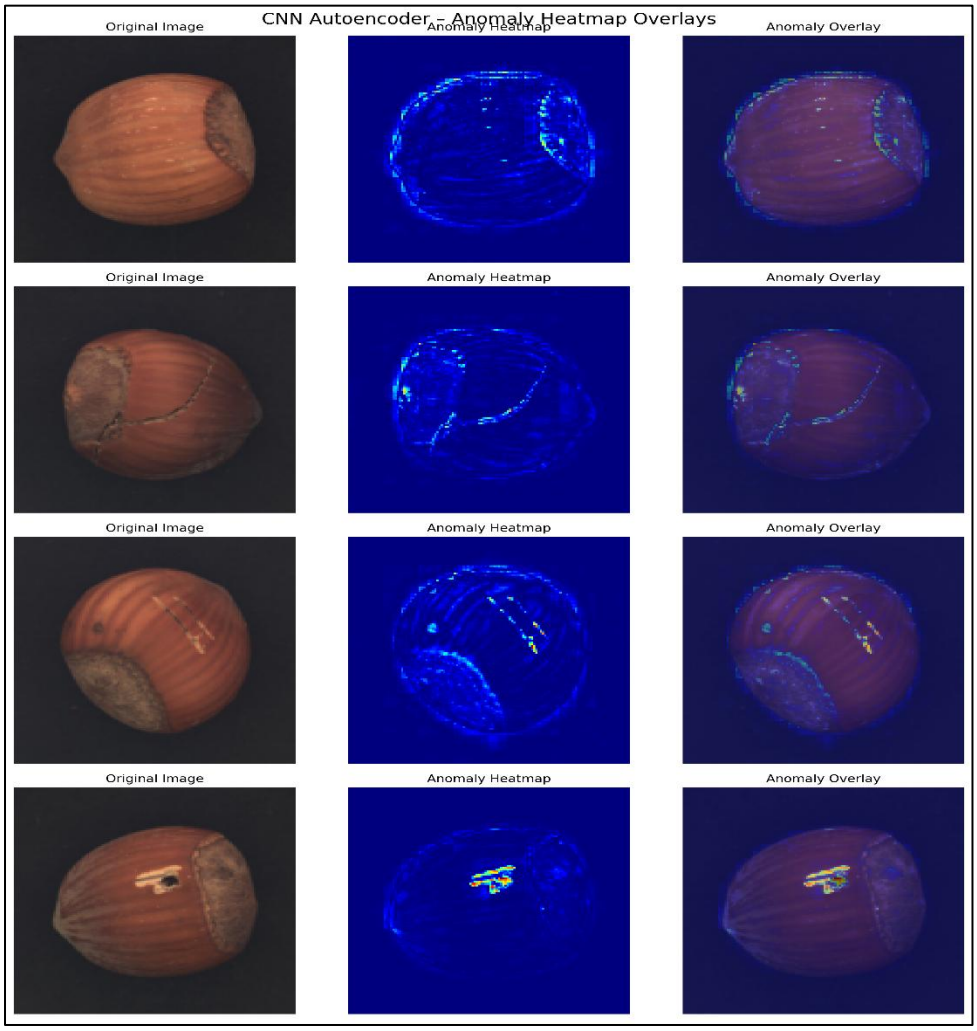
Criterion	**Reconstruction-error heatmap**	**LIME**	**SHAP**
Method type	Architecture-specific	Model-agnostic	Model-agnostic

Main explanation	Pixel-wise reconstruction errors	Local influence of image regions	Estimated contribution of image regions
Computational cost	Low additional cost	Relatively high	Potentially high
Main advantage	Directly connected to reconstruction-based anomaly detection	Explains individual predictions	Provides a theoretically grounded feature-contribution framework
Main limitation	Reconstruction error is not proof of a defect	Sensitive to segmentation and perturbations	Computationally expensive and dependent on feature representation
Implemented in this project	Yes	No	No

5.a.5 Justification of the Selected Method

The use of reconstruction-error heatmaps was based on their compatibility with the CNN autoencoder architecture as well as detection and localization of anomalies. In contrast to LIME and SHAP methods, reconstruction errors do not involve several model predictions but only use the reconstruction at inference, hence being more efficient in terms of computations and enabling integration into the Streamlit interface. Heatmaps provide a way for production managers and quality control inspectors to locate areas with high reconstruction errors without having to understand feature attribution numbers. Nevertheless, the presence of such heatmap is a signal of unusual reconstruction and does not imply any defects, underlining the importance of human participation in the quality control process.

5.b



Implementation and Validation of XAI

5.b.1 Implementation of the Explanation Method

A reconstruction-error heatmap was implemented to explain the predictions generated by the CNN autoencoder. This method is suitable for the selected architecture because the model reconstructs its input image, allowing pixel-wise differences between the original and reconstructed images to be calculated.

The explanation is generated through the following steps:

1. The input image is passed through the trained CNN autoencoder.
2. The reconstructed image is compared with the original image.
3. The mean squared reconstruction error is calculated for each pixel across the RGB channels.
4. Higher reconstruction-error values are represented by more intense regions in the heatmap.

5. A pixel-level threshold of 0.013013083 is applied to produce a predicted defect-region mask.

The heatmap is normalised separately for visualisation purposes. This improves the visibility of regions with higher reconstruction errors, but the displayed intensity should not be interpreted as a calibrated probability of a defect.

5.b.2 Integration into the Web Application

The explanation feature was integrated into the Streamlit application developed in Question 4. After an image is uploaded and processed, users can examine the prediction alongside visual information about the reconstruction errors..This interface is intended for production quality inspectors and shift supervisors. It allows them to examine suspicious regions rather than relying exclusively on the model's final classification.

5.b.3 Sanity-Checking the Explanations Using Occlusion

For each image, a 16 × 16 pixel patch around the region with the highest reconstruction error was replaced with the mean value of the surrounding image region. The anomaly score was then recalculated to observe whether removing the highlighted information changed the model's output.

Test image	**Actual class**	**Original anomaly score**	**Score after occlusion**	**Change**
193.png	Normal	0.013955	0.013783	-0.000172
003.png	Defective (crack)	0.009634	0.009071	-0.000562

In both cases, the anomaly score decreased after the highlighted region was occluded. The larger decrease occurred in the defective crack example, suggesting that the occluded region contributed to the reconstruction error.

However, the changes were relatively small. Furthermore, occlusion can introduce artificial image patterns that may themselves affect reconstruction. Therefore, this experiment provides a limited sensitivity check rather than proof that the heatmap identifies the true causal reason for a prediction.

5.b.4 Analysis of Incorrect Predictions

Three incorrectly classified examples from the final test set were examined to understand how the model behaved and what the reconstruction-error heatmaps revealed.

Image	**Actual class**	**Predicted class**	**Anomaly score**	**Error type**

309.png	Normal	Defective	0.025936	False positive
005.png	Defective (crack)	Normal	0.017478	False negative
003.png	Defective (crack)	Normal	0.009634	False negative

The classification threshold was 0.0192041565.

5.b.4.1 False Positive – 309.png

The hazelnut image was incorrectly labeled as defective because of the anomaly score exceeding the decision threshold. The heatmap suggested that there were regions where reconstruction errors were higher, which means that some visual features were reconstructed improperly. This can be caused by the variability of textures and appearances of hazelnuts, as abnormal reconstructions don't necessarily mean the presence of defects

5.b.4.2 False Negative – 005.png

This image was considered normal even when there is a crack because the anomaly score was lower than the chosen threshold. Despite the presence of high reconstruction error in some regions as shown in the reconstruction error map, the anomaly score was not enough to predict a defective image. This means that the model cannot recognize defects with high reconstruction error.

5.b.4.3 False Negative – 003.png

This report is about a hazelnut that has an anomaly score of 0.009634, which is quite low, below the threshold for defect classification. The presence of local reconstruction anomalies, as shown by a heatmap, did not translate to a high enough anomaly score to classify the hazelnut as defective.

5.b.5 Limitations of the Explanation Method

Although reconstruction-error heatmaps provide a useful visual indication of unusual image regions, several limitations must be considered.

- **Not causal explanations:** The heatmap shows where reconstruction errors occur. It does not establish that these regions are the actual physical causes of a defect or the definitive reasons for the model's prediction.
- **Normal image variation:** Natural texture and appearance differences can generate high reconstruction errors, resulting in false positives.
- **Subtle defects:** Small or less prominent defects may produce relatively low reconstruction errors and remain undetected.
- **Threshold dependence:** The predicted defect-region mask depends on the selected pixel-level threshold. Different thresholds may produce different localisation results.

- **Visualisation limitations:** Normalising each heatmap separately improves visibility but makes heatmap intensities less suitable for direct comparison between images.

Overall, the implemented XAI feature improves the transparency of the anomaly-detection application by showing users where reconstruction errors occur. The occlusion experiments provide an initial check of the highlighted regions, while the analysis of incorrect predictions demonstrates that the explanations can also help identify limitations in model behaviour. Nevertheless, the heatmaps should support human inspection rather than replace it.

Question 6

6.a Application of a Pretrained Foundation Model

6.a.1 Model Selection and Justification

For investigating a new technique of anomaly detection in industry, the pre-trained model CLIP (openai/clip-vit-base-patch32) was chosen for hazelnut defect detection. Since CLIP relates images to the language description, its algorithm of analyzing image-text similarity allows detecting whether hazelnut is good or defective without fine-tuning. Such an approach provides a great comparison to the CNN autoencoder since both models provide their prediction based on the same images of hazelnuts. It is essential to mention that CLIP is a foundation model which neither generates images nor text.

6.a.2 Model Configuration and Implementation

The pretrained model was used without fine-tuning on the hazelnut dataset. Each image was processed using the CLIP image processor and compared against two text descriptions representing the normal and defective classes.The model produced similarity-based scores for the two descriptions. These scores were used to derive a defective-class score for image-level evaluation.The scores represent relative similarity to the selected text descriptions rather than calibrated probabilities of a defect.

6.a.3 Prompt Design and Validation

Since CLIP uses text descriptions to compare against images, prompt selection was an important part of the experiment. Three prompt pairs were evaluated using the validation dataset.

Prompt configuration	**Class**	**Text prompt**
Prompt 1 – Simple descriptions	Normal	"a photo of a normal hazelnut"

	Defective	"a photo of a defective hazelnut"
Prompt 2 – Industrial descriptions	Normal	"a high-quality hazelnut with a normal, intact surface"
	Defective	"a defective hazelnut with visible surface damage"
Prompt 3 – Detailed descriptions	Normal	"a close-up photograph of a good hazelnut without cracks, cuts, holes or surface defects"
	Defective	"a close-up photograph of a defective hazelnut showing cracks, cuts, holes or surface damage"

The prompts were compared using the validation dataset. Average Precision (AP) was treated as the primary metric, with ROC-AUC also considered.

Prompt configuration	**Validation ROC-AUC**	**Validation AP**
Simple descriptions	0.5516	0.4725
Industrial descriptions	0.8257	0.7948
Detailed descriptions	0.8833	0.8507

The detailed prompt pair achieved the highest validation AP and ROC-AUC. Therefore, it was selected for the final evaluationThe decision threshold was also selected using validation data by maximising the F1-score. The selected threshold was 0.4252.mportantly, the prompts and decision threshold were finalised using validation results before evaluating the model on the held-out test dataset.

6.a.4 Evaluation on the Held-Out Test Dataset

The selected CLIP configuration was evaluated using the same 94 held-out test images used for the CNN autoencoder. The same image-level primary metric, Average Precision, was used to support a fair comparison.

The final test results were:

Metric	**CLIP test result**
ROC-AUC	0.9201
Average Precision (AP)	0.8969
Precision	0.7045

Recall	0.8857
F1-score	0.7848
Accuracy	0.8191

Using the selected threshold of 0.4252, the confusion matrix was:

	Predicted normal	**Predicted defective**
Actual normal	46	13
Actual defective	4	31

The model correctly identified 31 of the 35 defective hazelnuts, while missing four defective samples. However, it also incorrectly classified 13 normal hazelnuts as defective.

These results indicate that the pretrained foundation model was able to distinguish between normal and defective hazelnut images without being fine-tuned on the project dataset.

6.a.5 Comparison with the CNN Autoencoder

The selected CLIP configuration was compared with the final CNN autoencoder using the same held-out test dataset and image-level evaluation metrics.

Metric	**CNN Autoencoder (Top-1%)**	**CLIP**
ROC-AUC	0.8877	0.9201
Average Precision (AP)	0.8659	0.8969
Precision	0.8214	0.7045
Recall	0.6571	0.8857
F1-score	0.7302	0.7848
Accuracy	0.8191	0.8191

CLIP had better performance than the CNN autoencoder in ROC-AUC, Average Precision, recall, and F1-score with both being equally accurate. CLIP was able to detect 31 out of 35 defective images but failed to detect four of them while the CNN autoencoder failed to detect 12 images. However, CLIP had 13 false positives while the CNN autoencoder had five false positives. It means that CLIP was more sensitive to defects while CNN autoencoder gives pixel-level error heat maps for defect localization while CLIP does not provide defect-region localization.

6.a.6 Computational Cost and Limitations

Both models were evaluated on CPU using the held-out test dataset. The measured average inference times were:

Model	Total inference time	Average time per image
CNN Autoencoder	7.59 seconds	0.0807 seconds
CLIP	25.11 seconds	0.2671 seconds

CLIP took approximately 3.31 times longer per image than the CNN autoencoder in this experiment.These measurements include different processing steps: CLIP requires image preprocessing through its processor, whereas the CNN receives the preprocessed image tensor. Therefore, the comparison represents the observed pipeline times rather than a perfectly controlled measurement of model computation alone.

Other limitations include:

- **Prompt dependence:** Prediction performance was affected by the wording of the text descriptions.
- **No localisation:** CLIP provides image-level similarity scores but does not generate defect masks or pixel-level anomaly heatmaps in this implementation.
- **No fine-tuning:** The model was used in its pretrained form, so it was not specifically adapted to the industrial hazelnut dataset.
- **Input resolution:** Images were initially resized to 128 × 128 pixels for the project dataset. CLIP's processor subsequently resized them to its model input resolution of 224 × 224 pixels. Consequently, details lost during the initial resizing could not be recovered.
- **Score interpretation:** Similarity-based scores are not calibrated defect probabilities, and the selected threshold is specific to the model and prompt configuration.
- **Computational requirements:** CLIP required more inference time than the CNN autoencoder in the CPU-based experiment.

The model was accessed through Hugging Face and executed locally in Google Colab after downloading the pretrained weights. No paid external inference API was used. Internet access was required to obtain the model and its associated files initially. After downloading, inference could be performed locally, although computational resources and electricity were still required.

6.a.7 Summary

The study revealed that CLIP was a pretrained vision-language model that was capable of carrying out the defect detection task in hazelnuts quite efficiently since it recorded an Average Precision of 0.8969 compared to the CNN autoencoder that recorded 0.8659. Despite the fact that CLIP was more efficient in the detection of defects than CNN, the latter is still more useful where faster inference and good localization are needed. It thus means that pretrained ANNs like CLIP can be used as an alternative to specific task ANNs.

6.b Comparative Analysis and Innovation Opportunities

6.b.1 Comparative Predictive Performance

The CNN autoencoder and pretrained CLIP foundation model were compared using the same 94 held-out test images. Average Precision (AP) was used as the primary evaluation metric to account for the imbalance between normal and defective samples.

Evaluation metric	**CNN Autoencoder**	**CLIP**	**Better result**
Average Precision (AP)	0.8659	0.8969	CLIP
ROC-AUC	0.8877	0.9201	CLIP
Precision	0.8214	0.7045	CNN
Recall	0.6571	0.8857	CLIP
F1-score	0.7302	0.7848	CLIP
Accuracy	0.8191	0.8191	Equal

The CLIP model performed better in terms of Average Precision (0.8969) as opposed to the CNN autoencoder (0.8659), while also having a better ROC-AUC, recall, and F1 score. Nonetheless, the two models exhibited equal accuracy. On the other hand, the CNN autoencoder performed better in precision (0.8214) compared to CLIP (0.7045) since it was able to classify more images as being defective. CLIP was able to detect 31 defective images out of 35 images, but there were 13 false positives, whereas the CNN detected 23 defective images with five false positives.

6.b.2 Inference Latency and Computational Cost

Both models were evaluated using CPU-based inference. Their measured execution times were:

Model	**Total inference time**	**Average inference time per image**

CNN Autoencoder	7.59 seconds	0.0807 seconds
CLIP	25.11 seconds	0.2671 seconds

CLIP required approximately 3.31 times longer per image than the CNN autoencoder.

This difference is relevant in industrial environments where images may need to be inspected continuously or at high production speeds. The CNN autoencoder offers a lower-latency option, whereas CLIP requires more processing time in the tested environment.

However, these measurements include different preprocessing operations. CLIP uses its image processor, while the CNN receives preprocessed tensors. Therefore, the results represent the observed end-to-end processing times rather than a perfectly controlled comparison of model inference alone.

Regarding monetary cost, neither implementation used a paid external inference API. Both were executed locally using CPU resources. Consequently, there was no direct per-prediction API charge in these experiments. Actual deployment costs would depend on hardware, electricity consumption, maintenance and infrastructure requirements. A precise monetary cost per prediction was not measured.

6.b.3 Training Data Requirements

The two approaches differ considerably in their training-data requirements.

6.b.3.1 CNN Autoencoder

The CNN autoencoder was trained specifically for the hazelnut anomaly-detection task using 313 normal training images. It learned to reconstruct normal hazelnut appearances and used reconstruction errors to identify potential defects.

This approach requires a representative collection of normal images from the target production environment. Its performance may also need to be reassessed if the product appearance, lighting conditions or production process changes.

6.b.3.2 CLIP Foundation Model

CLIP was used in its pretrained form without fine-tuning on the hazelnut dataset. Instead of requiring task-specific training, it compared images with natural-language descriptions of normal and defective hazelnuts.

This reduces the need to collect and label a large task-specific training dataset. However, its performance depends on the pretrained model's learned visual-language representations and the quality of the prompts. The validation results demonstrated that prompt wording had a substantial effect on performance.

Therefore, the CNN requires task-specific normal training data, whereas CLIP can be applied without additional task-specific model training, although validation data is still necessary to select prompts and decision thresholds.

6.b.4 Privacy and Dependence on Third-Party Services

Both models were executed locally after their required model files and dependencies had been downloaded.Local inference reduces the need to transmit potentially sensitive industrial images to external inference services. However, appropriate access controls and data-storage practices are still necessary.

CLIP has a greater initial dependence on third-party model distribution because its pretrained weights and associated files were obtained through Hugging Face. Once downloaded, inference can be performed locally without sending images to a third-party API.Neither approach eliminates the need to consider software dependencies, model updates, security and deployment maintenance.

6.b.5 Conditions for Deployment

The results suggest that the suitability of each approach depends on the operational requirements of the production environment.

6.b.5.1 Conditions Favouring the CNN Autoencoder

The CNN autoencoder suits those situations when rapid image processing, low rate of false positives, ability to identify defective parts, access to images of normal production, and local execution are required. The feature of reconstruction error heat maps provides an advantage by enabling inspectors to concentrate on certain areas rather than get image classification only.

6.b.5.2 Conditions Favouring CLIP

CLIP would be more appropriate when:

- Higher defect detection recall is important.
- The organisation wants to experiment with different natural-language descriptions of defects.
- Limited task-specific training data is available.
- Additional inference time is acceptable.
- Image-level classification is sufficient and precise defect localisation is not essential.

However, its higher false-positive rate in this experiment would need to be considered. In a production environment, excessive false alarms could increase manual inspection requirements and reduce operational efficiency.Neither model should be deployed as an

autonomous industrial quality-control system based solely on these experimental results. Further evaluation using larger, representative production datasets and actual operating conditions would be necessary.

6.b.6 Innovation Opportunity: A Foundation-Model-Assisted Quality Inspection System

A realistic opportunity is to extend the existing CNN-based inspection system using CLIP as a complementary defect-description and classification component.The CNN autoencoder could remain the primary inspection model because of its faster inference and ability to generate reconstruction-error heatmaps. CLIP could be introduced as a secondary analysis component when an image is flagged as potentially defective or when the CNN produces an uncertain result.

For example, a quality inspector could compare a suspicious image against a collection of text descriptions representing different defect types:

- A hazelnut with a visible crack.
- A hazelnut with a cut or surface damage.
- A hazelnut with a hole.
- A hazelnut with printing or surface irregularities.
- A normal hazelnut with an intact surface.

CLIP could calculate image-text similarity scores for these descriptions and provide an additional indication of the type of defect that may be present.

This would extend the existing system beyond binary normal/defective detection by supporting defect-category suggestions through natural-language descriptions. The CNN heatmap could still provide spatial information about unusual reconstruction regions, while CLIP could provide additional semantic context.

Such a system could support human inspectors by combining the CNN's localisation capability with CLIP's image-text comparison capability. However, this proposed extension has not yet been implemented or validated. Its reliability would need to be tested using labelled examples of each defect category before operational use.

6.b.7 Overall Conclusion

The comparative experiment demonstrated that the pretrained CLIP foundation model achieved stronger image-level ranking performance, with an Average Precision of 0.8969 compared with 0.8659 for the CNN autoencoder. It also achieved higher recall, identifying more defective hazelnuts.

However, the CNN autoencoder provided higher precision, fewer false positives, faster measured inference and pixel-level defect localisation through reconstruction-error heatmaps.The findings indicate that pretrained foundation models are not automatically superior to task-specific neural networks. Their value depends on the operational requirements, available data, computational resources and expected output.

For the current project, the CNN autoencoder remains a practical primary inspection model where speed and defect localisation are important. CLIP offers a promising complementary approach where higher recall and natural-language-based defect analysis are valuable.Combining both approaches could provide an opportunity to develop a more flexible industrial inspection system while retaining human oversight for final quality-control decisions.

Declaration of Generative AI Use

Generative AI tools were used during the development of this coursework as supporting tools for learning, technical guidance and report preparation.

Tool used: ChatGPT(OpenAI) /Copilot

Purpose of use:

- To clarify artificial neural network concepts, CNN autoencoder architectures, anomaly detection and evaluation metrics.
- To receive guidance on Python implementation, debugging and troubleshooting during model development.
- To discuss experimental approaches, interpret evaluation results and understand model performance.
- To assist with structuring, drafting and improving the clarity of sections of the coursework report.
- To support the interpretation and discussion of explainable AI methods and pretrained foundation models.

Student responsibility and verification:

The dataset preparation, model implementation, training, experimentation, evaluation, Streamlit application development and generation of experimental results were carried out by the student. AI-generated suggestions and explanations were reviewed and checked against the actual implementation and experimental findings before being incorporated into the coursework.

The pretrained CLIP model was separately used as part of the coursework's foundation-model experiment. Its model details, prompts and experimental results are documented in Question 6.

The student takes responsibility for the accuracy, originality and final content of the submitted work